

Spring Security overview

1 Understanding Spring Security architecture

Refer :

"day15_help\Spring Security\regarding spring security.txt"

"Security flow summary.txt"

Diagrams :

"day15_help\Spring Security\diagrams\servletfilter-1.png"

"servletfilter-2.png"

"spring security architecture.jpeg"

"day15_help\spring security\diagrams\spring security flow.webp"

"The Final Authentication flow-4.webp"

OR

"Ultimate Details of Spring Security Architecture.png"

Steps

1. Create spring boot application , w/o adding spring security dependencies

Try to access some end points (eg : view products , add / delete products ..)

Observation : all end points were un protected !

Refer - products_api

2. Add spring security starter dependency in pom.xml , update maven project

```
<dependency>
```

```
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-security</artifactId>
```

```
</dependency>
```

```
<dependency>
```

```
    <groupId>org.springframework.security</groupId>
```

```
    <artifactId>spring-security-test</artifactId>
```

```
    <scope>test</scope>
```

```
</dependency>
```

3. What are defaults offered by Spring security out of box ?

3.1 : All end points are automatically PROTECTED (SECURED)

i.e entire web app is under a lock (authentication lock).

Un authenticated user CAN NOT access any of the APIs.

3.2 It supplies default user name : user

3.3 Generates 128 bit randomly generated password on server console , which changes upon server restarts.

3.4 Supplies ready made login n logout options

(form based authentication)

3.5 For RESTful web services , spring security offers HTTP Basic Authentication scheme.

4. For the details of - Basic Authentication mechanism

refer - "day15_help\spring security\Regarding Basic Authentication.txt"

5. Replace randomly generated 128 bit password n username - user by , custom details , by adding it in the application.properties file

spring.security.user.name=

spring.security.user.password=

BUT is this really practical ?

6. Replace it by in memory user details manager.

Refer - "day15_help\spring security\in-memory\SecurityConfig.java"

6.1 Use this SecurityConfig class

6.2 Configure PasswordEncoder bean in Application class

This approach is good for understanding spring security flow , this is also not very practical , since we want to keep user credentials(encrypted) in DB.

7. Replace in memory user details service - by DB based authentication (DB based User Details , accessed using JPA)

7.1 DaoAuthenticationProvider is an AuthenticationProvider implementation class that uses a UserDetailsService and PasswordEncoder to authenticate a username and password.

Dependencies - UserDetailsService and PasswordEncoder

7.2 UserDetailsService is used by DaoAuthenticationProvider for retrieving a username|email and a password.

7.3 Spring Security provides readymade implementations of UserDetailsService using in-memory, JDBC and caching

BUT for JPA based UserDetailsService - a programmer has to provide the custom implementation.

Steps

7.1 Remove InMemoryUserDetailsManager bean from spring config file.

7.2 Add UserEntity n UserRole - under <entities> package.

7.3 Add UserRepository

7.4 Run UserRepositoryTest to add some users , along with encrypted password in DB

7.5 Create custom implementation class of o.s.s.c.userdetails.UserDetailsService

Implement the method

public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException

7.6 Create Custom implementation class of UserDetails n supply user details - username(email) , password , granted authorities

7.7 Add DTOs for user sign up , user service n it's implemetation class.

7.8 Write Integration Test case for service-dao-db integration

7.9 Add UserController n test sign up.

7.10 Test other end points.

4. Continue to JWT based authentication

(Replace Basic Authentication scheme by JWT based authentication)

JSON Web Tokens (JWT) have become a popular method (Industry standard) for securing modern web applications, especially RESTful web services.

JWTs allow you to transmit information securely between parties as a compact, self-contained, and digitally signed JSON object.

For the understanding of JWT n steps :

Refer - jwt.io

Refer : "day15_help\spring security\JWT\JWT Details.txt"

Refer : "day15_help\spring security\JWT\JWT Steps Upgraded.txt"

Ref eclipse project : spring_security3

Steps overview

1. Copy from the earlier project

2. Add new end point in UserController , for signin

URL - <http://host:port/users/signin>

Method - POST

request payload : Auth req DTO : email n password

resp payload : In case of success : Auth Resp DTO : msg + JWT token + SC 200

In case of failure : SC 401

2.1 Store user credentials (email n password) in UsernamePasswordAuthentication token

2.2 Invoke authenticate method of AuthenticationManager.

Method of AuthenticationManager -

Authentication.authenticate(Authentication authentication) throws AuthenticationException

In case of success it will return the Authentication object , containing verified credentials.

2.3 Now comes the important step of JWT creation

(Pause here....)

3. For JWT creation

3.1 Add these dependencies , for JSON web token generation in pom.xml

```
<!-- https://mvnrepository.com/artifact/io.jsonwebtoken/jjwt-api -->
```

```
<dependency>
```

```
<groupId>io.jsonwebtoken</groupId>
```

```
<artifactId>jjwt-api</artifactId>
```

```
<version>0.11.5</version>
```

```
</dependency>
```

```
<!-- https://mvnrepository.com/artifact/io.jsonwebtoken/jjwt-impl -->
```

```
<dependency>
```

```
<groupId>io.jsonwebtoken</groupId>
```

```
<artifactId>jjwt-impl</artifactId>
```

```
<version>0.11.5</version>
```

```
<scope>runtime</scope>
```

```
</dependency>
```

```
<!-- https://mvnrepository.com/artifact/io.jsonwebtoken/jjwt-jackson -->
```

```
<dependency>
```

```
<groupId>io.jsonwebtoken</groupId>
```

```
<artifactId>jjwt-jackson</artifactId>
```

```
<version>0.11.5</version>
```

```
<scope>runtime</scope>
```

```
</dependency>
```

3.2 Copy these JWT properties (secret key n expiration time in msec) in application.properties file

Generate secret key : <https://generate-random.org/encryption-key-generator>

#JWT Secret key for signing n Verification

SECRET_KEY=.....

#JWT expiration timeout in msec for a day : 24*3600*1000

EXP_TIMEOUT=86400000

3.3 Copy JWT Utils class

Contains Methods for :

Token generation

Validation

Extracting the claims : email (username : subject) n authorities

You can add one more custom claim , for storing user id (if needed in your project)

4. Test User sign in endpoint (on swagger /postman) to confirm token generation

5. Any time client tries to access , any secured end point :

REST client has to add a Bearer Token in request Header : Authorization

(Confirm with Postman : choose Auth : Bearer Token)

5.1 To intercept any request , to check for the token :

Add custom filter : JWT custom filter (extending from OncePerRequestFilter)

5.2 Mark it @Component - so that it can be injected later in Security Config class

5.3 Check for "Authorization" header (request.getHeader)

If it's not null n starting with Bearer : extract token

5.4 Validate the token

Use - JwtUtils 's validateJwtToken.

In case of successful validation , it will return the Claims in the JWT.

From the claims : Extract - username|email (subject of the token claims)

n extract authorities

(Use JwtUtils methods)

5.5 Wrap user details (username|email +granted authorities) in the :

UsernamePasswordAuthenticationToken (imple of Authentication)

API -

o.s.s.c..authentication.UsernamePasswordAuthenticationToken

(Object principal, Object credentials, Collection<? extends GrantedAuthority> authorities)

principal - email

credentials (pwd) - null

granted authorities.

5.6 Save above Authentication object in the spring sec ctx holder

SecurityContextHolder.getContext().setAuthentication(Authentication token);

5.7 Continue with filter chain : doFilter

6. In Security configuration class -

6.1 To change the session config policy to stateless n add custom filter before :

UsernamePasswordAuthenticationFilter

6.2 Configure Authentication Manager bean.

6.3 Conf

copy from ready made sec config class

7. Test accessing secure end points

How will you test from postman ?

1. URL : <http://localhost:8080/products/view>

NO authentication required , you should get SC 200 , since it's permit all.

2. URL : <http://localhost:8080/products/purchase>

SC 401 , Authentication required n customer role is mandatory.

3. First authenticate (In the project , refer to : UserAuthController)

URL : `http://localhost:8080/users/signin` , method = POST , Body --raw --json

 $\{$

```
"email" : "a2@gmail.com",
```

```
"password" : "2345"
```

}

These are the correct credentials for the customer role.

In the response , SC 200 n in the resp body , you will get this

 $\{$

```
"jwt":
```

"eyJhbGciOiJIUzUxMiJ9.eyJzdWUiOiJhMkBNbnWFpbC5jb20iLCJpYXQiOiE3MDE3OTI0MzgsImV4cCI6MTcwMTg3OTIzOCwiYXV0aG9yaXRpZXMiOiJST0xFX0NVU1RPTUVSIn0.3XzhKJ0WPkra5b4-BkOfA7rbXWHSgzOcO MDDJOdyplESTGcG6FMCfXwzvs6Qa5ao7PUQ8dCxQazbHUQzeKxhQ",

```
"msg": "User authentication success!!!"
```

}

The value of JWT may differ.

4. Now for accessing the secured end point ,

URL : URL : http://localhost:8080/products/purchase , method=GET

Instead of choosing "Basic Auth", choose "Bearer Token", from Auth

n paste the JWT(Json web token)

You should get : SC 200

Resp body : customer should be able to purchase products

5. Similarly , you can test other endpoints.

Link - <https://docs.spring.io/spring-security/reference/servlet/authentication/architecture.html>